

Vision-Only Autonomous Drone Racing on Borrowed Hardware: What Team Electric Fire Built for the AI Grand Prix Physical Qualifier, and Why It Did Not Fly

Bojro Das, Geneustace Wicaksono, Etienne Sasenarine, John Apessos, Grant Lin, Rocky Shao Team Electric Fire (Team 10), AI Grand Prix Physical Qualifier, Anduril LC3, Santa Ana, California, 15–22 September 2026

Draft of 22 September 2026. Author order is provisional pending the team’s agreement.

Code: [bojro/aigp-sim](#) (simulation and policy training), [bojro/aigp-perception](#) (gate perception), [Code-Red-Cables/AI_GP](#) (flight client and on-site work, private).

Abstract

We describe the system Team Electric Fire built for the physical qualifier of the AI Grand Prix, an autonomous drone-racing competition in which a supplied 8-inch quadcopter must fly two laps of a ten-gate course using only a forward camera and the flight controller’s inertial sensors, with no external positioning and no human input during a scored run. The work spans four months and three generations of approach: classical colour-and-geometry vision with hand-written planners on the virtual qualifiers; imitation learning from human laps; and, for the physical event, a reinforcement-learning racing policy trained in NVIDIA Isaac Lab against a plant model of the real aircraft, fed by a YOLOv8-pose gate detector fine-tuned on hand-labelled frames from the aircraft’s own camera. In simulation the final 40 Hz policy passed 15.3 gates per episode with perfect corner observations and 10.2 with a corner-dropout model calibrated on the real detector; a separately trained hover policy settled within 0.30 m in 85.5% of episodes at the control rate the aircraft actually supports. On the aircraft we measured, rather than assumed, the facts a sim-to-real transfer depends on: a 40 Hz ceiling on the shared 115200-baud telemetry link, a gyro reported in raw counts at 16.384 per degree per second, an inverted pitch convention, a non-linear rate curve that turned the shipped linear stick map into a loop gain rising from 0.62 to 2.40 across the working range, a barometer that reads a descent whenever the propellers spin, and a keypoint threshold that had never been set and was discarding two frames in three. None of this produced a scored run. The team’s four aircraft were lost over the flight days to a sequence of flight-controller wedges, ground strikes and two autonomous strikes, one into a wall and one into the ceiling, before an autonomous hover had ever been demonstrated. We report the methods, the measurements, and the failure chain in enough detail that the next team can start from the last thing we learned rather than the first.

1. Introduction

Autonomous drone racing is a compact test of perception, state estimation and control under time pressure. The best published systems reach human-champion performance [1], but they do so with motion-capture-grade state estimation during training, custom airframes with characterised dynamics, and months of iteration on one platform. The AI Grand Prix physical qualifier removes each of those

advantages by design. Teams are handed a fleet of four identical aircraft they did not build, a companion computer with no deep-learning framework installed, a flight controller whose configuration they may not fully control, a hall with no tracking system, and five working days.

This paper is the record of what one team did with that. It is written as a research report rather than a post-mortem because most of the work is reusable and most of the measurements are new: nobody had previously written down, for this aircraft, how fast its telemetry link is, what units its gyro speaks, or how its rate curve distorts a policy's commands. It is also written because the work did not succeed, and the reasons are specific, ordered, and mostly avoidable.

The contributions are:

1. **A perception pipeline** that bootstraps gate-corner labels from the gate's own geometry, uses a detector to propose where geometry cannot trace, and then replaces both with human labels once the self-scored accuracy is shown to be optimistic by a factor of six (Section 4).
2. **A simulation and training stack** with an explicit, hashed observation contract shared between the simulator and the aircraft, a plant model with measured mass and derived rate-loop gains, latency and vision-delay modelling, and a corner-dropout augmentation calibrated on the real detector (Section 5).
3. **Bench measurements on the real aircraft** that settle the control rate, gyro scale, axis conventions and stick mapping, each of which had been assumed wrongly by at least one runner before it was measured (Section 6).
4. **A documented failure chain** from a working simulator to an empty scoresheet, with the hardware incidents in order (Section 7), and the lessons we would apply next time (Section 8).

2. The competition, the course and the aircraft

2.1 Rules that shaped the design

The physical qualifier specification (VADR-TS-005) defines a valid run as two laps flown “without any intervention of human pilots”. Ranking is by the number of gates passed, tie-broken by elapsed time to the last valid gate, with the best result of the final two days counting. Each team is assigned four aircraft, maintained and charged by the organizers' repair desk; teams are not financially liable for damage. A training cage with manual piloting is available in addition to timed track slots. There is no motion capture, GPS or other external tracking anywhere in the building, which rules out every measurement procedure that assumes a ground-truth position reference.

The scoring rule matters more than it looks. Because gates count first and time second, a slow policy that passes two gates beats a fast one that passes one and crashes. The team's plan on the last day was built around that: not a lap, but “passing one or two gates from the competition start pad”.

2.2 The course

Ten identical gates stand inside an 85×165 ft boundary (25.9×50.3 m). Each gate is a square orange annulus, 2.7 m outer, 1.5 m opening, with its opening centre about 1.35 m above the floor. Gate 9 is a stacked double gate whose upper opening (about 4.05 m) is flown southbound and lower opening northbound, so a lap is 11 crossings and a run is 22. The organizers publish the gate coordinates; the team's simulator track was digitised from a drawing and turned out to be 6.5% too small with two headings off, which Figure 1 shows and Section 5.4 quantifies.

PQ course: official coordinates vs team Isaac track
arrows = required flight direction; labels = opening-centre height

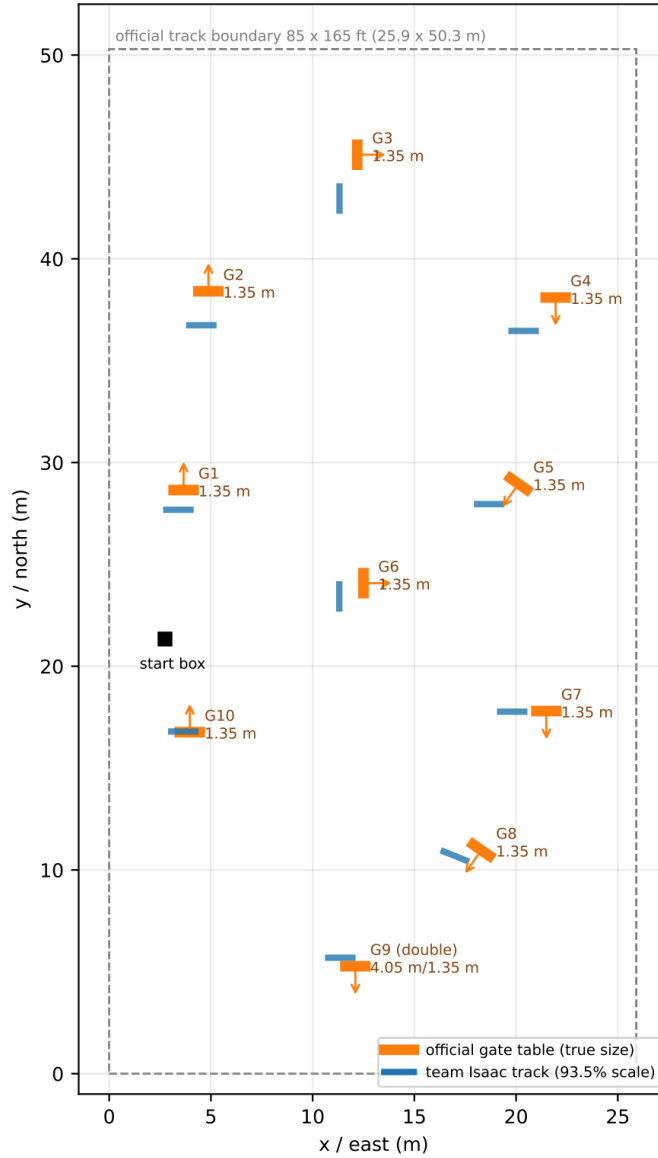


Figure 1. The official gate table (orange, true size, with required flight direction and opening height) against the team's original Isaac track (blue), which had been digitised from a picture at 93.5% scale. The start box is on the floor, 7.4 m before gate 1.

2.3 The aircraft

The supplied aircraft is a Neros Archer B2: 8-inch propellers at 4.1 pitch, 1745 g all-up race weight (weighed on site, 17 September), a 6S pack, four DShot300 motors with bidirectional RPM telemetry, and a Betaflight 4.4.3 flight controller on an STM32H743 with an ICM42688P gyro and a DPS310 barometer. The companion computer is an NVIDIA Jetson Orin NX 16 GB on a Sseed A603 carrier at a 25 W power budget, running an image with OpenCV (GStreamer build), NumPy and pyserial and, by the organizers' design, no PyTorch or TensorRT in the system Python. The camera is a 12.3 MP Arducam IMX477 with a rolling shutter and an M12 lens, mounted with a 20° upward tilt that the team set and locked. We used it at 1920×1080 and downscaled to 640×360 for inference; three independent calibrations put the horizontal field of view at 73.7–74.1°, a focal length of about 425 px at 640 px width, against the specification's nominal 75°.

The Jetson talks to the flight controller over a single 115200-baud UART using the MultiWii Serial Protocol (MSP). Both directions share that link: telemetry requests from the Jetson, and the stick stream it sends when flying. The consequences of sharing appear in Section 6.1.

2.4 The handover mechanism

Betaflight’s *MSP override* lets a companion computer replace selected radio channels with values it streams over MSP while an auxiliary switch is held. On the team’s aircraft the override mask was 15 (roll, pitch, throttle, yaw), the override mode was on AUX5 with its down position engaging the policy, ARM stayed on AUX1 under the pilot, and ANGLE (self-levelling) on AUX2. Three facts about the firmware, read from the 4.4.3 source and then confirmed on the aircraft, shaped every runner the team wrote:

- there is **no timeout** on MSP stick data, so a process that stops streaming leaves the flight controller flying the last frame it received, indefinitely;
- the MSP stick buffer starts at **zero**, which the receiver logic treats as a dead radio (RX_FAILSAFE after 0.3 s, motor cut after 5 s, and the AUX channels freeze so the override switch itself can no longer be seen), so valid frames must be streaming before anyone touches the switch;
- the override mask has **no MSP route** and can only be set from the Betaflight command-line console, which on this board wedged the flight controller twice out of two attempts on 18 September and was banned unconditionally from then on.

3. System overview and the three generations of approach

Every autonomous racer runs the same loop: find the gates in the image, estimate your own motion, decide, actuate. The team implemented that loop three different ways over the summer (Table 1), and the physical-qualifier system inherited pieces from each.

Period	Approach	Outcome
Jun 2026	Organizers’ example client on the virtual qualifier; HSV colour mask for the orange opening; PnP range; attitude-rate controller over MAVLink	Passed the first gate on 6 June; a waypoint replay completed the course “sub 14 seconds” using the simulator’s own position
Jul 2026	Virtual qualifier 2 removed position telemetry. YOLO corners + IMU dead reckoning (Q2_pnp, passed gates 1–2); a 16-state dual-gate EKF (Q2_ka lman); a DreamerV3 world model (168k steps, zero gates, deleted)	No autonomous lap; human personal best 35.96 s
Aug 2026	Li & de Croon style classical stack; a snake-gate detector; HG-Dagger imitation of 17–18 human laps with a temporal-convolutional policy	Human best 14.04 s; the imitation policy “leaves the pad and will crash”
30 Aug–16 Sep	Isaac Lab PPO on a digitised copy of the physical course; keeper checkpoint pq_speed_best at 33 gates per 40 s episode	Fast but brittle (Section 5.4); abandoned on the corrected simulator
17–22 Sep	On site: MSP adapter, bench measurements, hand-labelled gate detector, 40 Hz retrains with dropout, three hover runners, a classical “stack_min” fallback in simulation	No scored autonomous run; fleet lost (Section 7)

Table 1. The eras of the project. The branch names in the archived repository match the middle column.

The physical-qualifier system works as follows. The camera produces frames at 30 fps; a YOLOv8n-pose detector places the eight corners of each visible gate; the corners, with roll, pitch and body rates from the flight controller and a dead-reckoned body velocity, are packed into a fixed-width observation history; a NumPy multilayer perceptron exported from the trained checkpoint maps that history to a collective-thrust value and three body rates; an adapter turns those into four RC channel values through the inverse of Betaflight’s rate and throttle curves and streams them at 40 Hz. A gate tracker maintains which gate is next, because nothing on the real course publishes it. A separate classical stack (`stack_min`) used the same detector and a perspective-n-point pose, an alpha-beta position filter and a stop-and-go guidance cascade in ANGLE mode, as the slow fallback.

4. Perception: eight corners of an orange square

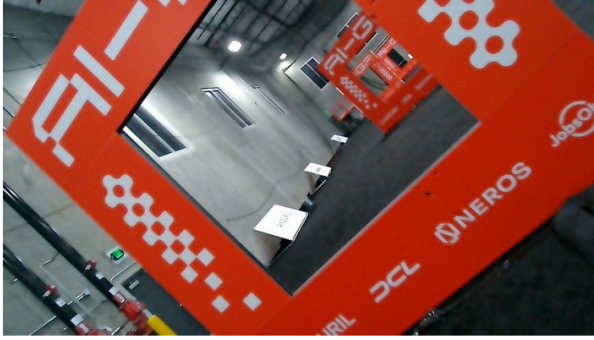
4.1 The problem

The gate is a planar orange ring. Its eight keypoints (four outer, four inner, clockwise from top-left, with a fixed left-right flip index) give a perspective-n-point pose once at least four are visible. The detector must run inside a 33 ms budget on the Orin, cope with the gate filling and overflowing the frame as the aircraft closes, and not confuse the printed sponsor banners and signage that share the gate’s colour. When the team arrived on site, no detector had been trained on real gate images: the existing models had been tuned on simulator renders, and the only footage of a real gate was one teammate’s phone.

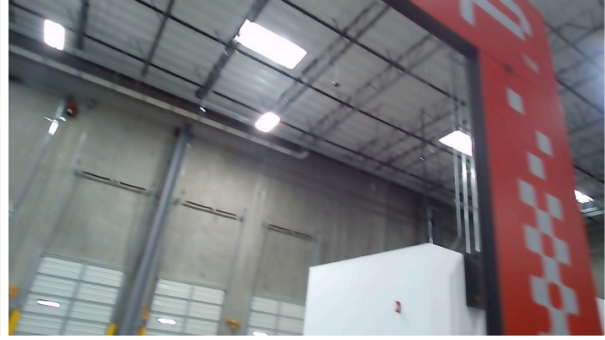
4.2 Data

On 19 September a teammate carried the aircraft’s own camera around a gate in the hall, producing 1207 frames at 2 fps (two sessions, 1920×1080) with hardware capture timestamps. A further 331 frames were captured from the aircraft near a gate on 20 September, including close-ups where the ring runs past the frame edge (Figure 2b). No flight footage exists; every training image is a person walking.

(a) onboard camera, hand-carried walk-around, 19 Sep



(b) onboard camera at a close gate, 20 Sep: corners leave the frame



(c) geometric auto-label: eight numbered corners



(d) hybrid label as written for YOLO-pose training



Figure 2. What the onboard camera sees. (a) A walk-around frame from 19 September. (b) A frame from the aircraft on 20 September at a close gate; the corners the policy needs are outside the image. (c) The geometric auto-labeller’s output, eight numbered corners. (d) The same gate as a hybrid label written for YOLO-pose training.

4.3 Geometric auto-labelling

Hand-labelling eight ordered corners on a thousand frames was not available on the first day, so the team wrote a labeller that measures the gate rather than recognising it. A colour mask of the ring has a hole; in the contour hierarchy the parent is the outer square and the child hole is the opening, so ring identity comes from topology rather than from telling eight similar corners apart. From there it is measurement: fit lines to each side, intersect consecutive sides for corners (which recovers corners past the frame edge), refine each side to the image’s own colour step at sub-pixel resolution, and fit one homography to all eight points, which is over-determined and pulls both rings onto a geometrically exact gate. Each corner is marked visible only if both of its sides were measured. Frames whose largest orange blob yields no label are quarantined rather than written as negatives, so a missed gate cannot become a “no gate here” training example.

On the 1207-frame capture the labeller accepted 1026 gates across 816 images and sent roughly a third of instances to review. Its self-scored accuracy, the median offset of a refined corner from the image’s own edge, was 0.75 px.

4.4 The hybrid, and why confidence cannot replace geometry

A teammate’s detector, trained earlier on Roboflow-labelled frames, missed 0 of 127 geometrically verified gates on frames it had never seen but placed corners to about 3.8% of gate size; the geometry placed them to 0.2% but only where it could trace them. So the detector proposes and the geometry disposes: each proposal is refined to the image’s edges and put through the same per-corner checks the labeller applies to itself. That recovered about 170 extra gates per 200 frames at the same accuracy, taking the accepted total from 1026 to 1917. The detector’s own confidence could not have done this: recovered proposals scored 0.53 and rejected ones 0.56, no separation at all.

Fine-tuning yolov8n-pose on the hybrid labels, scored on held-out contiguous blocks of frames, raised box mAP50-95 from 0.283 to 0.636 and pose mAP50-95 from 0.760 to 0.854, with recall on verified gates going from 98% to 100% and keypoint error against the geometry from 11.8 to 8.3 px (Figure 3, left). Simulated motion blur to about 215°/s of yaw barely dented recall.

The split is not a detail. Frames come half a second apart along a walked path, so neighbours are near-duplicates and a random train/validation split scores the model on what it memorised: the same model reads 0.517 box mAP50-95 on a random split and 0.283 on contiguous blocks (Figure 3, centre). Every earlier accuracy claim in the team, including a reported 0.90 mAP, had used a random split.

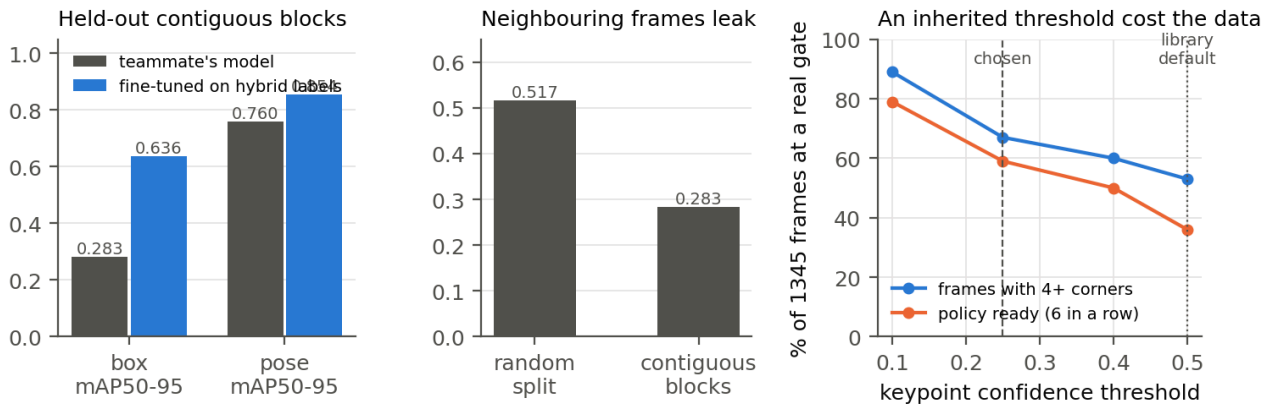


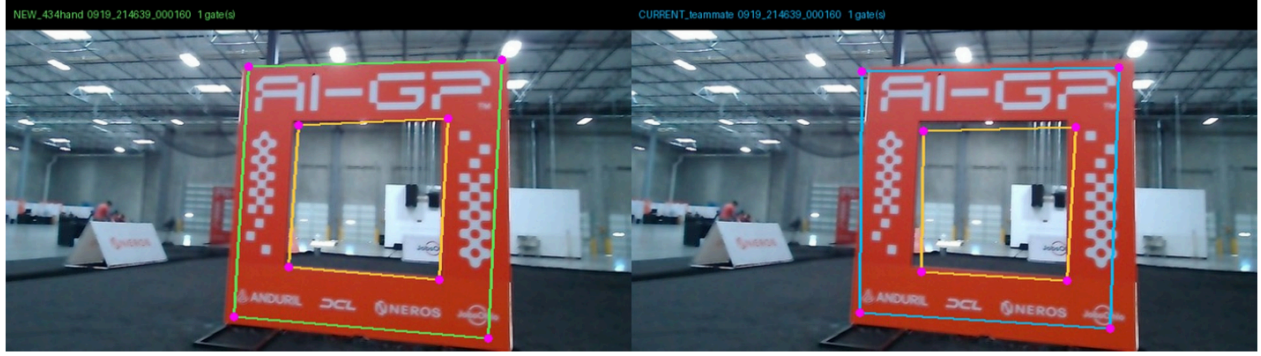
Figure 3. Left: the teammate's model against the fine-tune, on held-out contiguous blocks. Centre: the same model scored two ways. Right: the keypoint confidence threshold the runner inherited from the library (0.5) against the value chosen at a real gate (0.25), measured over 1345 frames; the gate itself was detected in every frame at every setting.

4.5 Human labels, and the number that was wrong

Once a Roboflow export of human annotations arrived (340 frames, 737 gate instances at first, later 497 frames), the self-scored accuracy could be checked. Against humans the labeller's corners were **4.36 px median, 9.64 px at the 90th percentile**, not 0.75 px. The 0.75 px figure had been measured against the gate's own geometric model, which is the thing the labeller solves for, so a systematic error fits perfectly. Some of the difference is a person clicking a corner at 1920×1080; the true labeller error sits between the two, nearer the top. Sub-pixel was never a claim the team could make, and the documents were corrected to say so.

Two models were trained on the human labels, gate_pose_hand434 and gate_pose_hand497 (150 epochs, batch 64, seed 0, vertical flips disabled because a vertical flip silently swaps top and bottom ring identity). On 67 held-out human-labelled frames the hand-labelled model's close-gate recall (gates larger than 30% of the frame) was 100% (32/32) against 71.9% (23/32) for the incumbent; the incumbent also stacked nested duplicate boxes on a single close gate, seven detections for two gates in one checked frame, which is a flicker source no temporal filter can remove because whichever duplicate wins non-maximum suppression changes from frame to frame (Figure 4). Three silent data faults had to be fixed before hand497 could be trained at all: Roboflow splits three ways and the builder read only the training split (115 of 497 labels would have been discarded); 58 labelled frames lived in a different capture directory; and macOS tar had written 2617 AppleDouble ._* files that Python's glob would have scanned as corrupt images.

(a) a typical frame: hand-labelled model (left) and the teammate's model (right) agree



(b) a selected worst case: the incumbent stacks seven boxes on two gates

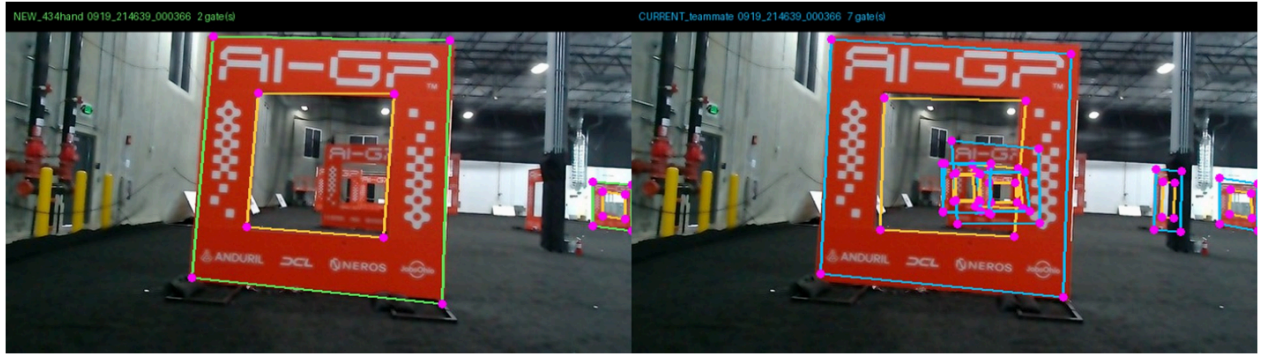


Figure 4. Side-by-side on the same frames: the hand-labelled model (left panes) and the incumbent (right panes). (a) A typical frame where both find the one gate. (b) A selected worst case, not a fair sample: the incumbent reports seven gates.

4.6 On the aircraft

On the Orin, inference at 640×360 took 27.4–27.6 ms per frame for either model on the GPU at the 25 W profile (36 fps; FP16 changed nothing measurable), with the whole capture-to-corners producer loop at 35.4 ms and a mean packet age of 47 ms as seen by a 40 Hz consumer. Running two models on one frame costs about 61 ms, so the always-both dual-detector mode (Figure 5) could not fly, and the side-by-side comparison on the Orin was a viewing tool rather than a flight configuration.

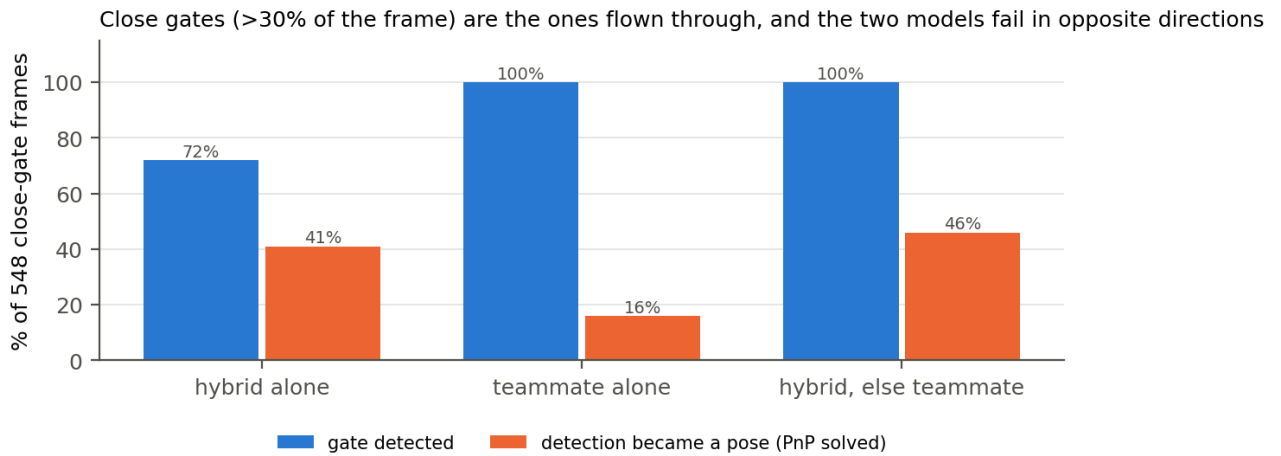


Figure 5. On the 548 close-gate frames of the walk-around capture (laptop CPU, 640×360): the hybrid model is near-blind to 28% of close gates but poses what it sees; the teammate's model sees every gate and returns corners too rough to solve; the combination beats both.

Two findings at a real gate on 21 September mattered more than the model choice. First, the keypoint confidence threshold below which a corner is reported invisible had never been set; the library default of 0.5 was flying. The runner needs four visible corners and wipes its six-frame history on any frame that falls short, so at 0.5 the policy could act only 36% of the time. At 0.25, checked by eye, 59% (Figure 3, right). Second, comparing hand434 and hand497 over 1345 frames at that threshold, hand497 flickered 30% less (4.2% against 6.0% per-corner flip rate between frames), broke into half as many usable fragments (37 against 78) and had a longer unbroken run (178 against 142 frames). Those two per-corner statistics, a 13% dropout rate with 0.81 stickiness, became the dropout model the racing policy was retrained against (Section 5.6).

Two further fixes applied to any model flown. The reprojection-error cap of 2.0 px in the pose solver was right for the simulator’s exact corners and rejected 74% of the gates the model found on real frames, against 33% at 8 px, with no increase in range jitter. And the aim point, computed as the average of whichever corners were visible, slid onto the frame instead of the hole as soon as the inner corners left the image, 11–47% of the gate’s width off, exactly as the aircraft closes.

5. Simulation and policy training

5.1 The stack

The training stack is a fork of Kousheek Chakraborty’s `isaac_drone_racer` on Isaac Sim 4.5 and Isaac Lab 2.1 with skrl 1.4.2 PPO [5, 6, 7]: 4096 environments, 120 Hz physics, a policy step every second physics step (60 Hz) or every third (40 Hz), a shared three-layer 256-unit ELU trunk with a Gaussian head, 24-step rollouts, five learning epochs, adaptive learning rate on a 0.01 KL threshold, and running standardisation of observations and values. A full 50 000-timestep racing run at 4096 environments takes about 48 minutes on an RTX 6000 Ada and costs under a dollar; the constraint on this project was never compute.

5.2 The observation contract

Two implementations of one vector exist: a batched torch builder in the simulator and a NumPy builder on the aircraft. Keeping them separate was deliberate; forcing both through one implementation would make both worse. What must never differ is the definition, because if the two disagree by a channel order or a clip, nothing raises: the policy receives numbers whose meaning drifted and flies into a gate with complete confidence. The repository therefore defines the contract once, in dependency-free Python, and hashes every value both ends must agree on; the aircraft-side assembler was verified to reproduce the simulator’s 1632-number history exactly, with projection agreeing to 3.9×10^{-6} px, and the NumPy policy runtime matches PyTorch to 2.5×10^{-6} .

The per-frame vector, version 1, is 51 channels: 16 normalised pixel coordinates of the target gate’s eight corners (−1 when unseen), 8 visibility flags, gravity-referenced roll and pitch, three body rates clipped to ± 8 rad/s, three components of a commanded body velocity, an 18-slot one-hot of which gate is next, and a lap fraction. Thirty-two frames are stacked, giving 1632 inputs. Version 2 appends the four actions the policy last issued, $55 \times 32 = 1760$. Version 2 exists because of latency: with no delay a policy can infer what it commanded from what happened next, so action history is redundant; with 40 ms of real delay it is not, and the published ablation the team relied on [8] measured trajectory tracking going from 10/10 to 0/10 when action history was removed with delay still simulated.

Three choices in that vector are worth defending. The velocity is *commanded*, dead-reckoned from thrust, attitude and a drag model, never integrated from the accelerometer, because the accelerometer on this airframe is biased low by about a third of a g at full throttle through vibration. The corners are latched at the camera’s 30 Hz and then delayed by one to four control steps per environment, so the policy sees perception the way the aircraft delivers it. And the action fed back is the *issued* command, not the delayed one, because the aircraft knows what it sent the instant it sends it; feeding back the delayed command would return the delay to the policy as free information.

5.3 The plant

The upstream simulator modelled a 0.6 kg 5-inch racing quad. The physical aircraft is $2.9\times$ heavier with arms about $2.2\times$ longer, and the naive fix, setting the mass, changes nothing, because thrust in the simulator is normalised by mass and cancels. What does not cancel is thrust-to-weight, rotational inertia and drag. The corrected plant sets a mass of 1.745 kg distributed at startup across the airframe and four propeller links (the first attempt applied 1.745 kg to every link, an 8.7 kg aircraft with a thrust-to-weight ratio of 0.2, caught by the smoke test); an estimated inertia of (0.0040, 0.0055, 0.0075) $\text{kg}\cdot\text{m}^2$, written along with a corrected centre-of-mass orientation after the read-back revealed a 7.16° authoring tilt in the asset that coupled roll into yaw; rate-loop gains derived from that inertia and a target time constant rather than inherited (the inherited gains made the corrected aircraft 84% slower in pitch and produced 73% of crashes as gate strikes); a thrust curve pinned to 1 g at a per-episode hover stick drawn from 0.21–0.29 (the top end is a drained pack) and a per-episode thrust-to-weight from 4.3 to 8.0; a zero-to-two-step action delay; and a first-order lag of 8–45 ms on the rate setpoint, matching the flight controller’s measured 15 Hz setpoint smoothing and the roughly 40 ms command-to-actuation latency others have reported for Betaflight [9]. The rate limit of 3.2 rad/s at full action was kept because it is what the aircraft-side decoder expects.

5.4 What the first policy taught us

The keeper checkpoint from the pre-event era, `pq_speed_best`, lapped the team’s simulator track in 7.4 s at a mean of 14.4 m/s and a peak of 25.4 m/s, 59 gates in a 40 s episode with no crash (Figure 6). Figure 7 shows the lineage of runs that produced it from the surviving TensorBoard logs, and the continuation run one day later that scored exactly zero gates for 50 000 timesteps.

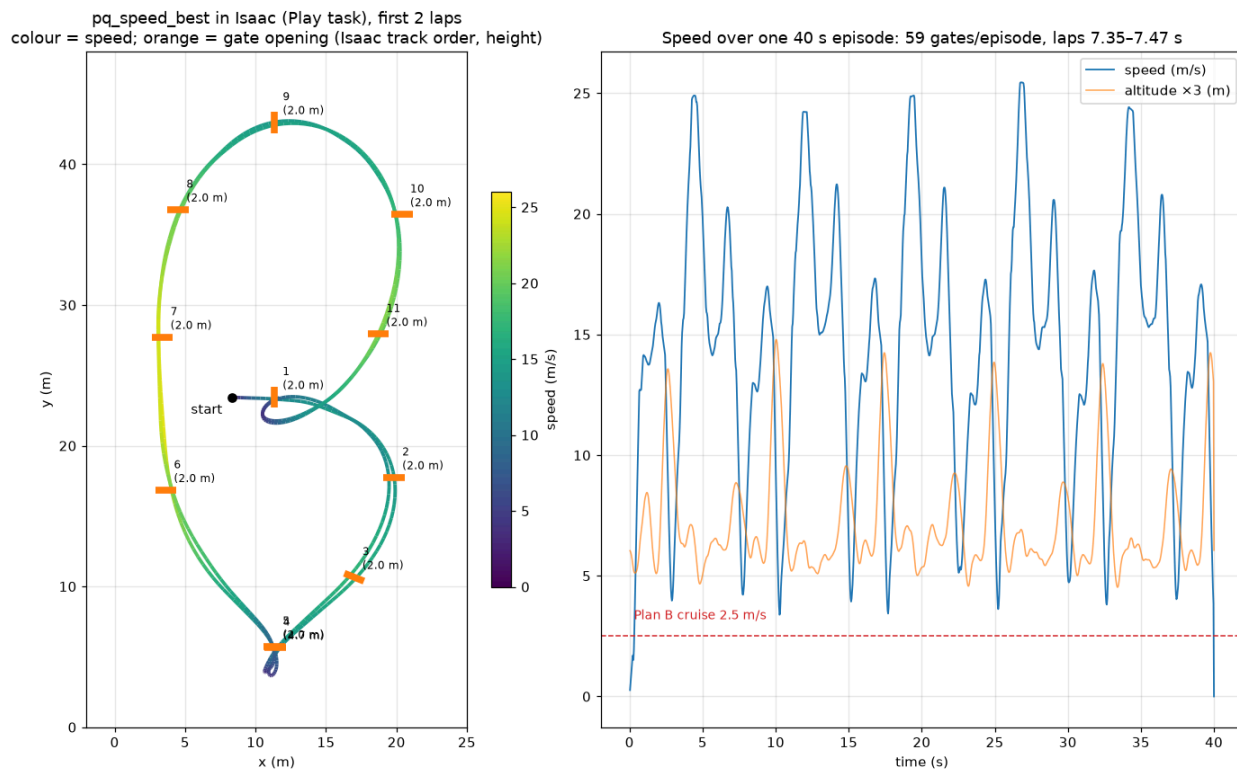


Figure 6. pq_speed_best in playback on the team's original track: path over two laps coloured by speed, and speed and altitude against time. The dashed line is the fallback plan's 2.5 m/s gate speed for comparison.

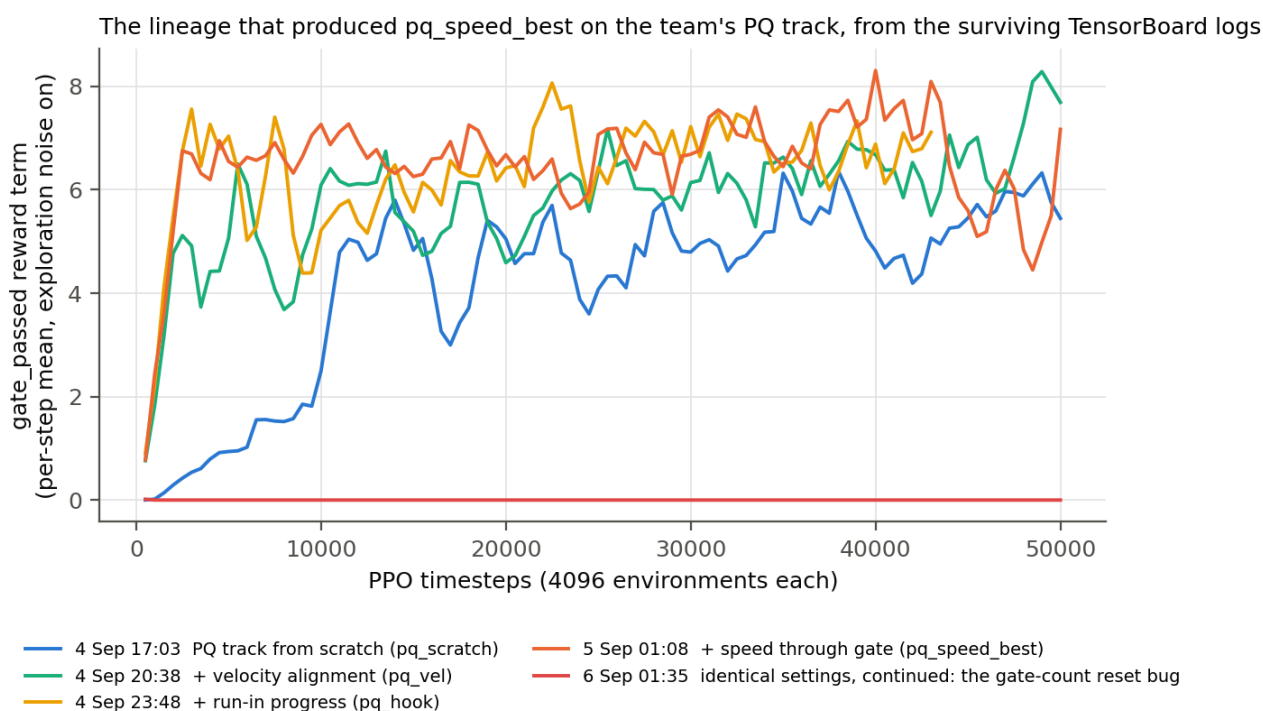


Figure 7. The lineage on the team's PQ track, 4–5 September, and the continuation on 6 September that passed no gates. The continuation was the first symptom of a reset bug: whenever any environment reset, the previous-position buffer was overwritten for all environments, so no crossing could ever be registered again. A one-line fix confirmed by an A/B test restored the gate reward from 0.01 to 1.1.

That zero was a bug, but the 59 gates were not a result either. A stress study on 16 September ran the policy on 512 aircraft under one change at a time (Figure 8): on its own track it crashed 1.7 times per 100 gates; with the real camera's 74° field of view instead of the 90° the code assumed, 42; with 50 ms of total delay, 55; with hover thrust 10% weaker than assumed, 58. Scaling the track by the 7.5% the digitisation had lost raised crashes from 2 to 16, with 60% of them on the climb to the double gate's upper opening: the policy had learned a manoeuvre timed to the distances it trained on. On a mirrored or random course it crashed almost immediately (73 and 47), which is expected of a policy whose input names the next gate, but it confirms it learned the course rather than a skill. On the corrected simulator with the real camera it passed four gates and crashed, every time.

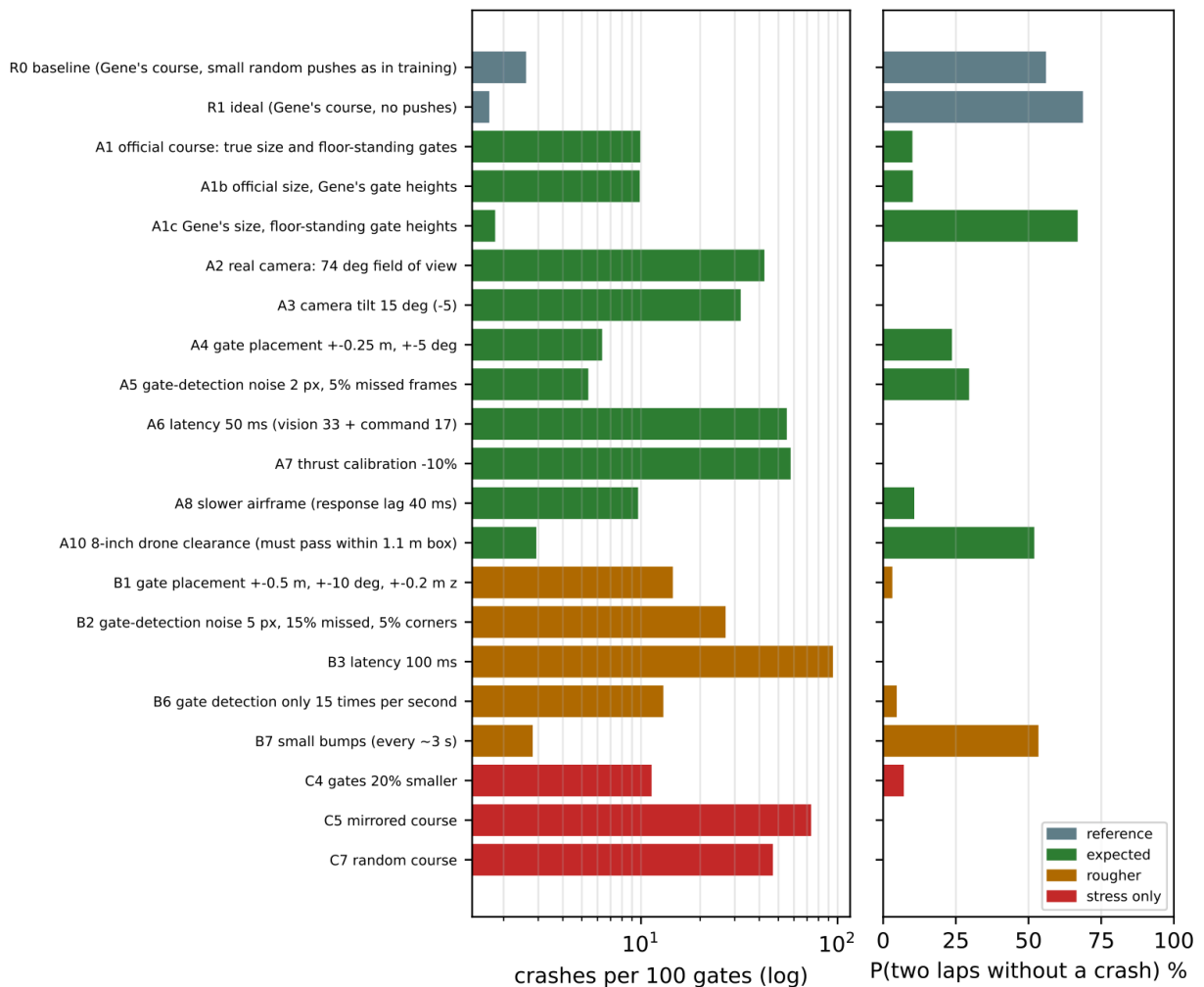


Figure 8. Crashes per 100 gates (log scale) and the implied chance of two clean laps for `pq_speed_best` under single perturbations, grouped by how likely each is at the real event. Latency and thrust calibration dominate; the camera model is next.

A one-hour fine-tune on the corrected simulator with the expected disturbances randomised got through 6.3 gates per attempt with 11 crashes per 100 (Figure 9), ten times better under the conditions it trained on, and worse than the original with no disturbances at all, because it had learned to expect delay and drag. That was the evidence for renting a GPU and retraining from the plant model of Section 5.3 rather than patching the old policy.

continued training of Gene's model on the corrected simulator (official course, 74 deg camera, expected disturbances randomized), 2048 simulates

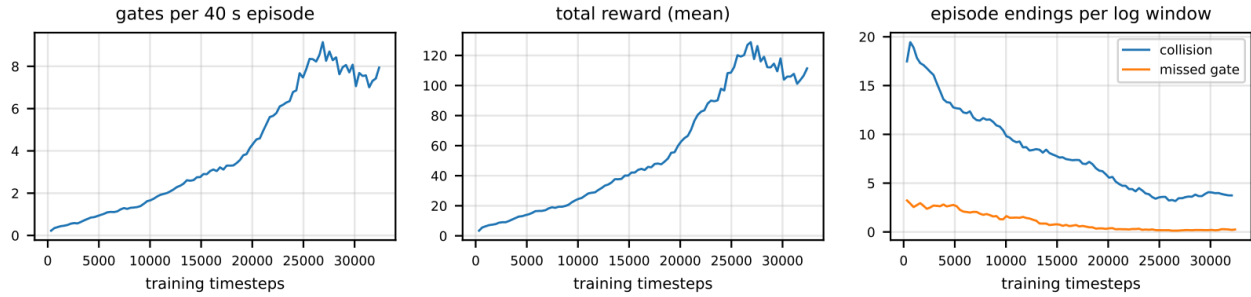


Figure 9. Training curves of the one-hour fine-tune on the corrected simulator: gates per 40 s episode, mean total reward, and episode endings by cause.

5.5 Rewards, starts and the smoke test

The racing reward is dominated by 600 points per gate passed and 20 per metre of progress toward the aim point, with a -100 termination penalty, small terms for looking at and centring on the gate, and an over-speed penalty above 8 m/s. The speed cap was added when the corrected-plant policy settled at a 14 m/s median and 21 m/s at the 95th percentile with 73% of its crashes being gate strikes; two laps of 106 m need only 5.3 m/s to finish in 20 s. The hover task, a subclass, rewards approaching a 2 m standoff in front of a gate, station-keeping, and being *settled* (within 0.30 m and under 0.30 m/s), and terminates 8 m from the target.

Starts turned out to matter as much as rewards. The best racing policy averaged 9.5 gates but, in 456 attempts from the real start position on the floor 7.4 m before gate 1, passed gate 1 zero times: it had never trained a takeoff. Adding pad and floor starts exposed two silent bugs in sequence, a spawn pose snapshotted before the code that set it (0 of 1024 spawns near the pad although 20% were configured) and a height jitter applied after height selection that put 15% of floor spawns underground. After the fixes, 256 pad starts passed gate 1 86.3% of the time with a median 2.15 s. A fifteen-check smoke test now gates every training run on a rented box; its own exit code had been reporting success while failing, because Isaac Sim's shutdown hard-exits before Python's `sys.exit` runs and discards the buffered output.

5.6 Results at the rate the aircraft supports

Section 6.1 shows the link supports 40 Hz, not the 60 Hz the policies trained at. Figure 10 quantifies the cost: the best 60 Hz hover checkpoint settles in 87.0% of episodes at its trained rate and 52.6% run at 40 Hz (with the 95th-percentile excursion going from 0.37 to 0.93 m); pinning the delay to zero attributes 22.6 points of that to the rate itself and 13.5 to the extra latency. The 60 Hz racing policy keeps 37% of its gates at 40 Hz. Retrained at 40 Hz, the hover policy settles 85.5% with a 0.331 m 95th-percentile excursion, a better tail than the 60 Hz policy managed at its own rate, and still 89.0% if run at 60 Hz.

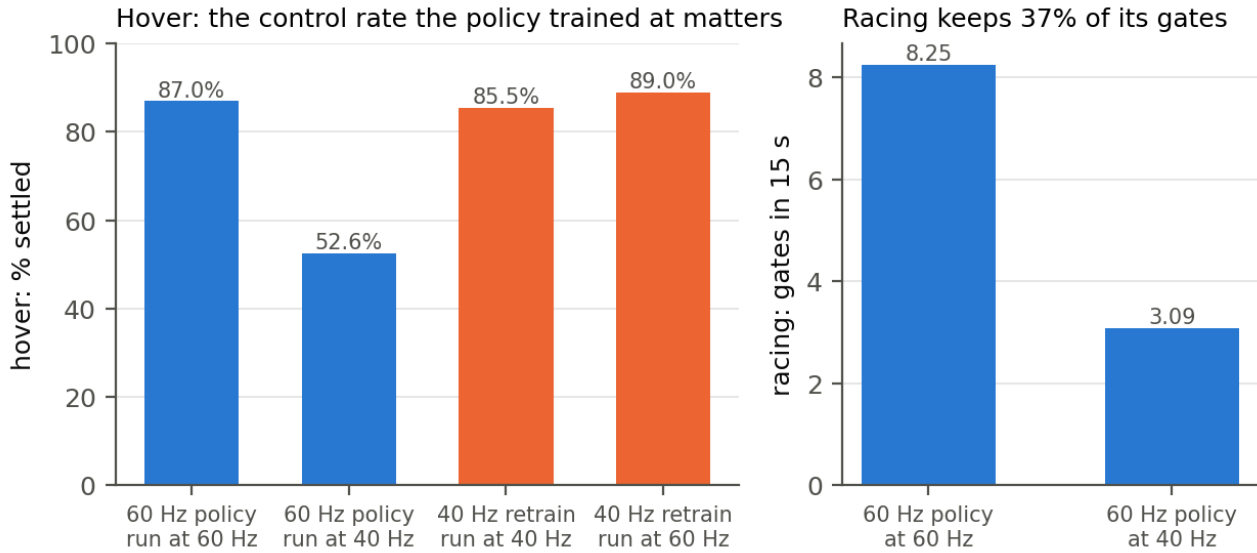


Figure 10. The control rate is part of the plant. Left: hover settled fraction for the 60 Hz checkpoint and the 40 Hz retrain, each run at both rates. Right: the 60 Hz racing checkpoint’s gates in 15 s at both rates.

The hover policy’s robustness sweep (Figure 11) ranks what the aircraft could get wrong: it is robust to how the aircraft *responds* (a slower rate filter, a lighter mass) and fragile to how much thrust it *needs*. A four-step command delay, the direct proxy for running the 60 Hz policy at 40 Hz, is the worst non-thrust condition at 13.0% settled. During this work the hover policy’s exploration noise was found to be *growing* while racing’s shrank, because once hover converges the task gradient goes quiet and the entropy bonus, amplified by an adaptive learning rate whose default ceiling is 100× the base rate, becomes the dominant pressure; hover got its own optimiser settings and its settled fraction went from a 68.3% peak to 77.4% and then 87.0%.

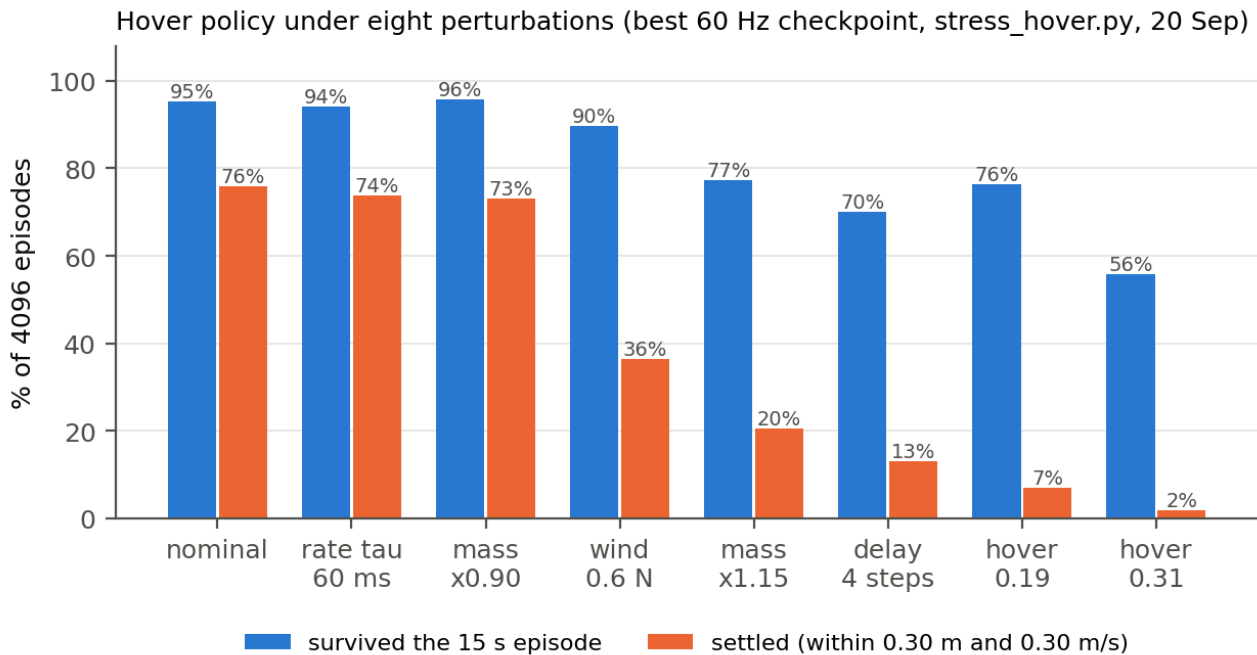


Figure 11. The hover policy under eight perturbations, 4096 episodes each. The hover-stick rows are outside the trained band and outside what the aircraft does; they bound the failure mode rather than describe a live risk.

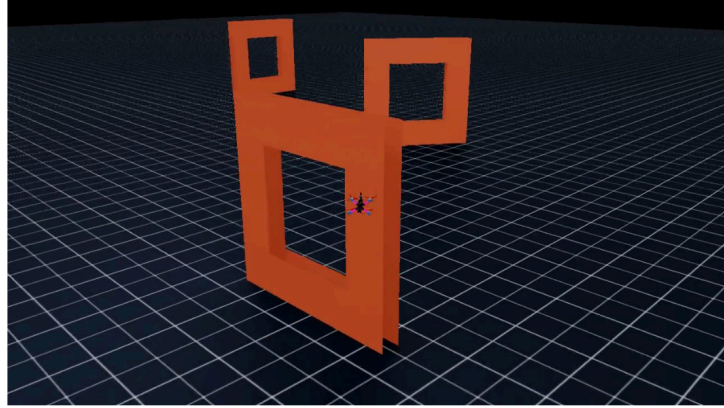
Corner dropout is modelled as a two-state Markov chain per corner with the measured 13% drop rate and 0.81 stickiness (Section 4.6). Applied at evaluation only, it costs the racing policy 71% of its gates; independent per-frame dropout at the same rate would leave the policy “ready” 0.5% of the time where the clustered real process leaves it ready 24%, so the correlation is the mercy. Trained against it, the `race40drop` chain reached 10.23 gates per episode *with corners dropping* against `race40`’s 15.27 with perfect corners, and in the one head-to-head run from the competition pad under measured dropout scored 9.31 gates to `race40`’s 4.62. Writing dropped corners as zero rather than the “not seen” sentinel collapsed racing from 12.8 to 0.13 gates per episode, a reminder that the sentinel is part of the contract.

Figure 12 is what the policy’s own camera sees during a simulated run, and Figure 13 links the recordings: [race_start_from_pad.mp4](#) is the takeoff from the competition pad that Section 5.5’s start fixes made possible, and [race_best.mp4](#) is the best racing checkpoint from a chase camera.

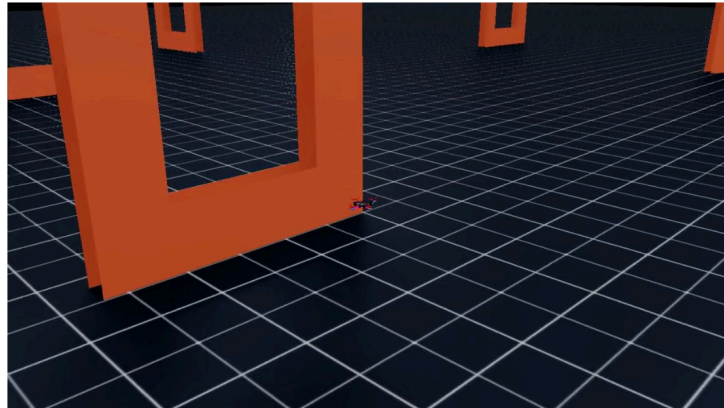


Figure 12. Contact sheet from the onboard camera in Isaac during a racing episode, one tile every 0.5 s for 20 s.

(a) race_start_from_pad.mp4: the 40 Hz racing policy lifts off the competition pad and takes gate 1



(b) race_best.mp4: the best racing checkpoint, chase camera



(c) stack_run_02.mp4: the classical stack, chase view (left) and its own camera with detections (right)

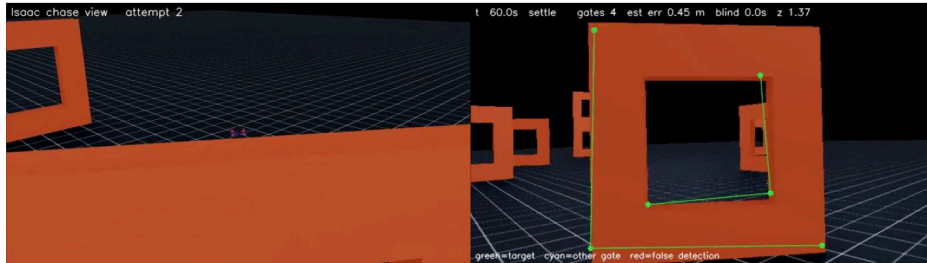


Figure 13. Frames from the three recordings in paper/videos/. (a) [race_start_from_pad.mp4](#), 6.7 s: the racing policy starts on the floor 7.4 m before gate 1 and flies through it. (b) [race_best.mp4](#), 9.9 s: the best racing checkpoint on the corrected course. (c) [stack_run_02.mp4](#), 60 s: the classical fallback stack of Section 7 in Isaac, chase view on the left and the drone's own camera on the right with the simulated detector's corners drawn (green target gate, cyan other gates, red false detections) and a status line of time, phase, gates passed, estimation error and time blind. Runs [01](#) and [03](#) are the other two attempts.

6. The aircraft, measured

Everything in this section was measured props-off, never armed, without the Betaflight console, on 18–21 September. Each item overturned something a runner had assumed.

6.1 The link runs at 40 Hz, and one message is load-bearing

Read-only, the telemetry snapshot (attitude, raw IMU, status and RC in one batched MSP_MULTIPLE_MSP request) round-trips in 10.0 ms and looks good for 97 Hz. That number is a mirage: telemetry and the stick stream share one UART, and contention scales with the stream rate (Figure 14). With a 40 Hz

stream the 95th-percentile snapshot is 20.3 ms against a 25 ms budget; with a 60 Hz stream it is 30.3 ms against 16.7 ms. The wire itself accounts for 8.5 ms of the 10 ms floor; the rest of the degradation under load is the flight controller’s MSP task scheduling, not bandwidth, and raising the baud rate is stored configuration and off limits. The batched message had never been exercised by anyone before this: neither the organizers’ library nor their fake flight controller implements it. Without it, four separate requests cost 66.9 ms at a 40 Hz stream, fail the runner’s own timing gate at every rate, and at their worst exceed the 60 ms after which the runner rejects the observation. If a firmware change ever drops message 230, the aircraft does not get slower; it stops flying.

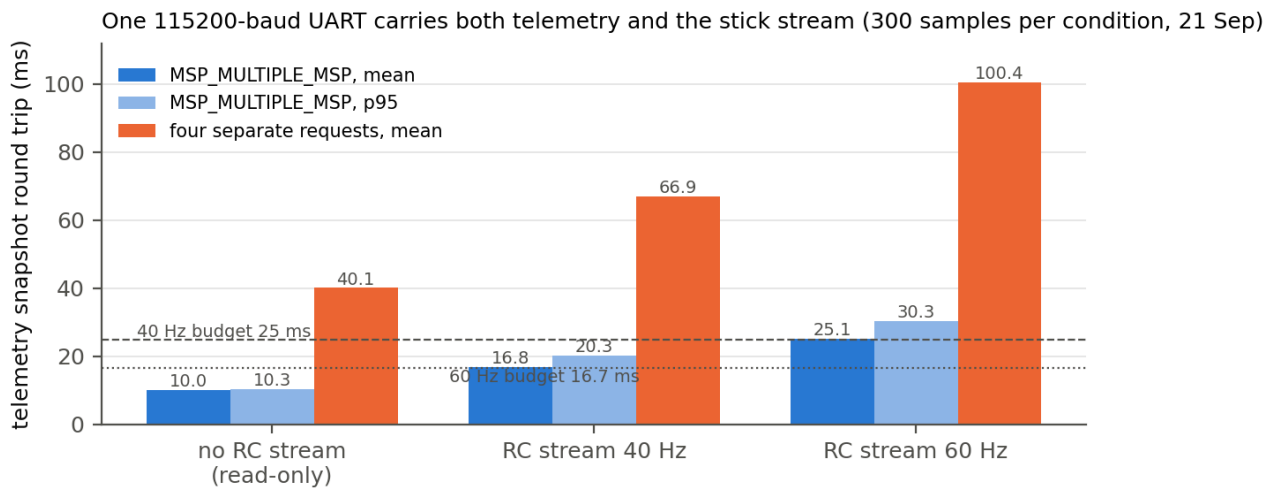


Figure 14. Telemetry snapshot latency with and without the stick stream, batched against the four-request fallback. 300 samples per condition, measured on the aircraft over USB.

6.2 The gyro speaks in counts

MSP_RAW_IMU returns raw 16-bit gyro counts, not degrees per second. Rotating the airframe by hand and integrating the gyro against the flight controller’s own attitude gave ratios of 16.14 on pitch and -16.25 on yaw, against $32768/2000 = 16.384$, the scale of a 16-bit gyro at $2000^\circ/\text{s}$ full scale. A hover log from 18 September had already shown a peak of 4894, impossible as degrees per second and unremarkable as counts. The deployed runner converted the counts with `np.radians()` directly; since the policy clips its gyro channel at $\pm 8 \text{ rad/s}$, a real rotation of $28^\circ/\text{s}$ saturated the observation. The organizers’ library documents the value honestly as raw units; the comment in their IMU check script (“gyro already in deg/s ”) is wrong for this build, and neither of its checks can catch it: near-zero at rest passes either way, and a $30^\circ/\text{s}$ response threshold passes more easily when the number is inflated sixteenfold.

6.3 Axis conventions

Stick readings on 21 September: roll right 2005 and left 1000; yaw right 2005 and left 997; pitch **nose-down 2005** and nose-up 997, reversed from the usual RC convention. The flight controller labels nose-down positive on attitude, gyro and stick; the simulator’s attitude channel labels nose-up positive. Every runner written before 21 September carried the pitch sign the other way. The final adapter negates pitch on both the command and the observation, and one teammate reached the same conclusion independently the previous night from the organizers’ own docstring. The two flight-controller messages disagree with each other about yaw: the heading rose turning right while the yaw gyro integrated negative.

6.4 The rate map, or why the first policy flight ended in a wall

The deployed runner mapped body rates to sticks linearly, sending the policy's full ± 3.2 rad/s envelope to full stick. Reading the aircraft's rate profile over MSP: rc_rate 55, super rate 75, expo 0, so full stick commands $440^\circ/\text{s}$ on roll and pitch and $380^\circ/\text{s}$ on yaw, while the policy's envelope is $183^\circ/\text{s}$, and the curve is not a line. Figure 15 shows the result: the loop gain is 0.62 at small commands, crosses unity mid-range and reaches 2.40 at full stick. A small correction returns two-thirds of what was asked, so the policy pushes harder; past the crossover it is handed $2.4\times$. Under-respond, over-correct, diverge. The replacement adapter inverts the curve by bisection over the monotonic forward model, which on this profile (expo 0) is exact. The throttle map was, by contrast, right: the whole hover band falls inside one segment of the 12-point throttle curve with a local gain of 0.914, an 8.6% constant that the policy's thrust-to-weight randomisation already covers.

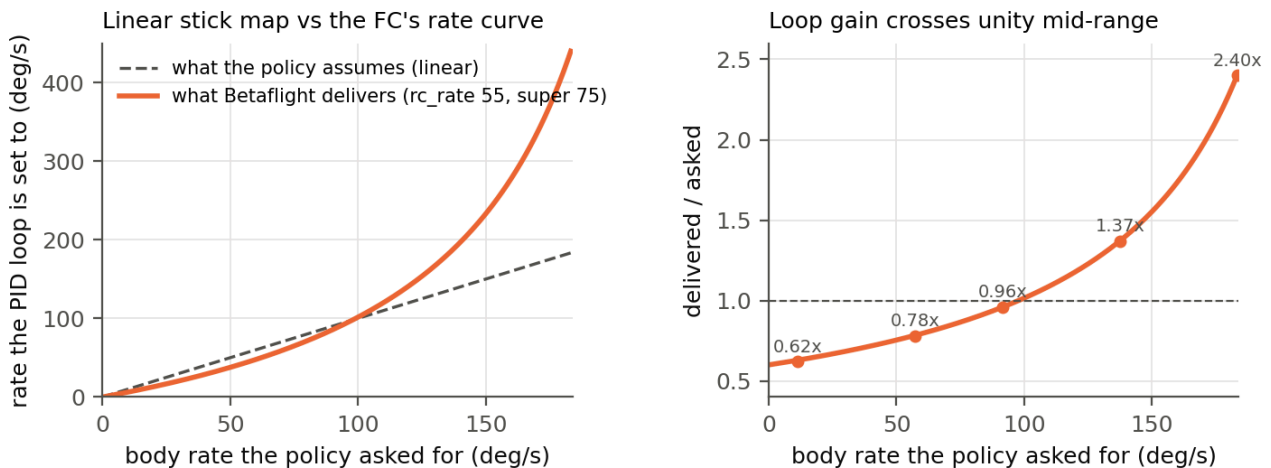


Figure 15. What the shipped linear stick map delivered against what the policy assumed. Every runner on this stack validated commands to the same 3.2 rad/s envelope, so the fault applied to every policy, not one.

6.5 The barometer, and what the first logs were worth

The four hover logs from 18 September were invalid as hover measurements: one reported a hover throttle of 1098 μs (9.8%), which would imply an aircraft that lifts ten times its weight, and was the drone armed on the floor. They were worth keeping for three findings. The barometer falls 1.6–3.4 m whenever the propellers spin (Figure 16) and its vertical-speed output read exactly 0.000 in all 2828 samples, so the logger's stability gate filtered nothing; the gyro is in counts (above); and the logger recorded the stick, not the motor outputs, so the post-curve throttle the motors actually saw is unrecoverable. Height therefore had to come from the camera, and every hover design depended on a gate being in view.

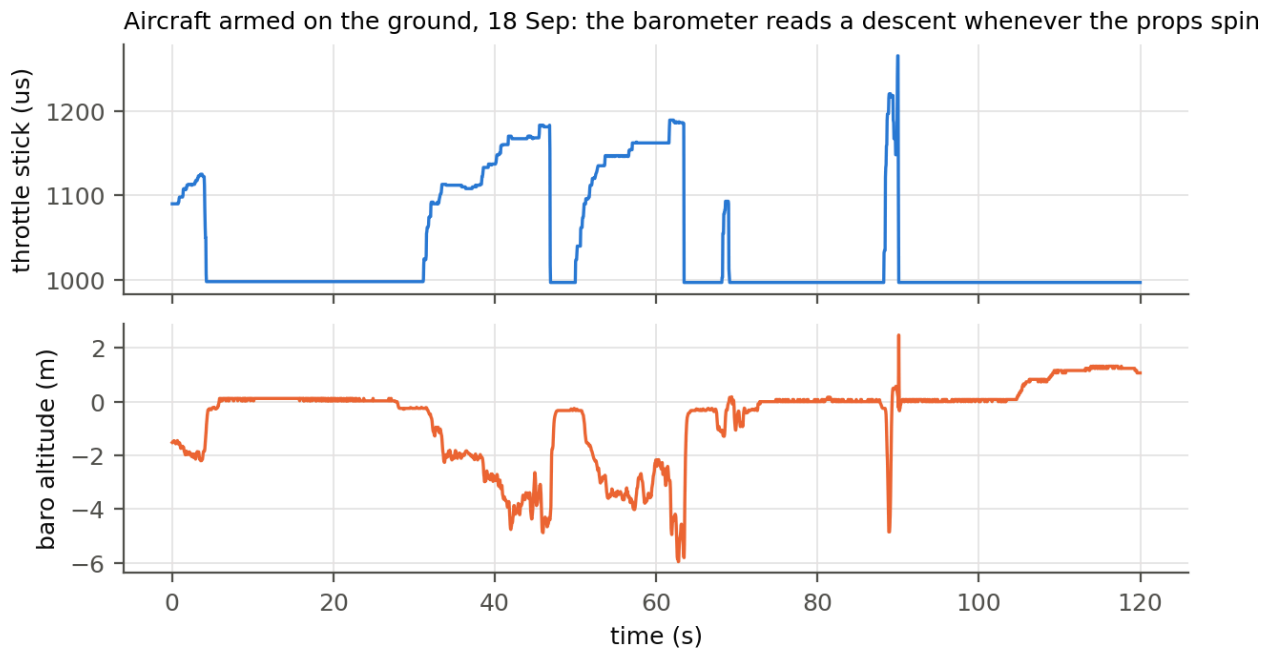


Figure 16. Throttle stick and barometric altitude over two minutes with the aircraft armed on the ground. Each throttle rise reads as a descent of several metres.

6.6 What the aircraft did with a policy in shadow

A shadow mode runs the whole chain, camera to policy, and logs the command it would have sent while transmitting nothing (a test parses the source to prove the send path is structurally unreachable). At the hover pose on 21 September the runner obtained 15 usable observations from 1262 frames before the keypoint-threshold and history-gap fixes; the 20° upward tilt hides both bottom corners at that pose, so the minimum visible corners was lowered from four to three. Earlier, a static observation with four visible corners had produced a confident $1636\ \mu\text{s}$ throttle command against a hover estimate of $1233\ \mu\text{s}$. A degraded view does not produce a degraded command; it produces a confident wrong one.

7. Flight attempts and the loss of the fleet

The repository does not contain a scoresheet for 21–22 September, and the last commit of the event is timestamped 01:18 UTC on 22 September. What it records, in order, is a bring-up that never produced an autonomous hover and a fleet that did not survive the attempt. The team’s own account is that all four aircraft were unflyable by the end.

1. **18 September.** The Betaflight console wedged the flight controller twice in two attempts, each needing a battery pull; a profile write was lost mid-wedge. Flipping the transmitter switch labelled ACRO/ANGLE “made the drone die”: the mechanism, confirmed from source, is the override engaging on a zero stick buffer. Every manual attempt that day was in ACRO, because ANGLE was assigned to no switch, at a throttle too low to leave ground effect; the logs show tip-overs and a 3.3 g ground strike followed by the firmware’s runaway-takeoff protection disarming the aircraft. Nobody on the team could hand-hover for more than a second. The airframe was replaced that evening, voiding the day’s camera calibration.
2. **19 September.** The flight controller went silent again, on MSP and to the transmitter, and was recovered by hand; the console ban became unconditional. Plan A, the trained policy, was

deferred in favour of a slow self-levelled hover-then-one-gate plan. Motors ran with no radio bound, props off.

3. **20 September.** A Betaflight reset occurred and the configuration owner wrote a restore procedure for a human to run. The battery was found at 2.55 V per cell. The first props-on policy attempt ran 0.45 s: twenty policy samples with the decoded collective rising from 0.54 to the 0.90 maximum before the pilot disarmed. The rate-map audit that evening (Section 6.4) is titled “Why the first policy flight ended in a wall”.
4. **21 September.** The flight controller hung with the configurator and the Jetson both attached; the aircraft came up armed twice with the ARM switch already down. The fallback stack’s first flight “hit the ceiling”: with the gate out of frame its takeoff trim integrated the height error and pinned the throttle at its cap, climbing at 1.0 m/s against a 0.4 m/s design. The venue asked for extra care. A replacement Jetson whose OpenCV lacked the GStreamer camera path appeared that night. All reinforcement-learning chains were stopped and the classical stack, which reached 95% three-gate success in a simulator it was tuned in (Figure 17), was chosen for the final day; there is no record of it flying.

Twelve simulated attempts of the perception stack from the pad (plan view): blue = true path, dotted = the stack’s own position estimate, X = crash

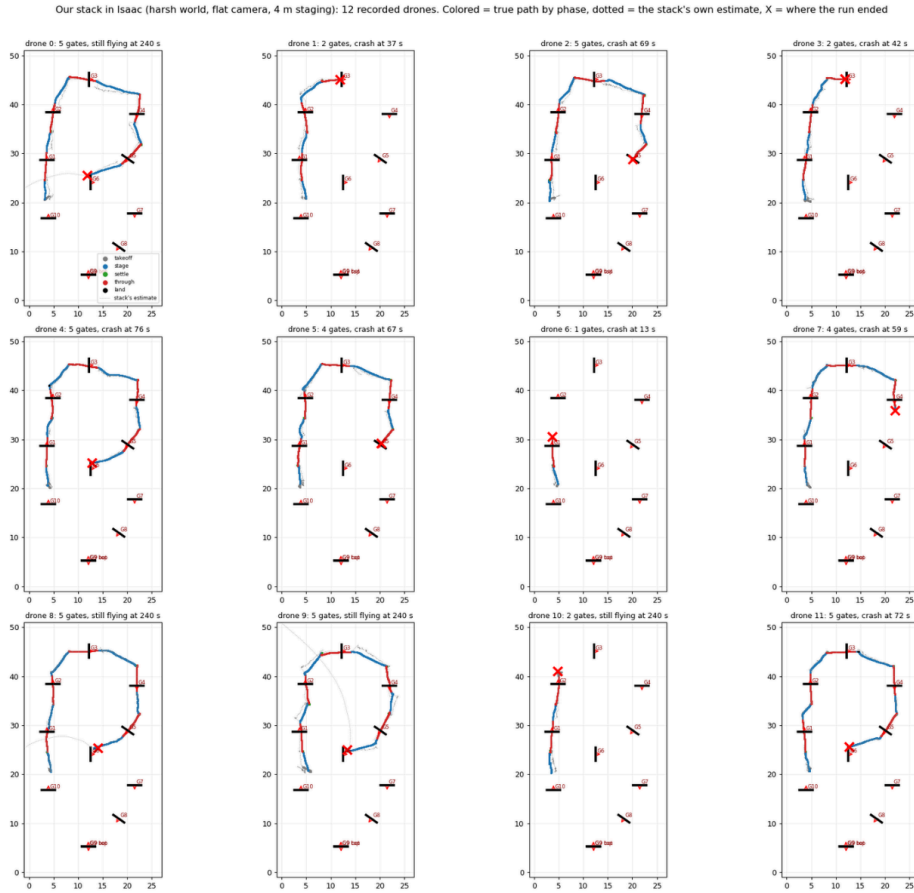


Figure 17. The classical fallback stack, flown in Isaac against a pessimistic sensor model (every gate detected, gate-shaped false positives, sticky corner dropout, 60 ms latency, an unknown self-levelling gain). Most crashes fall between gates 5 and 6, where the course reverses.

The facts that would have had to be true for a scored run, and were not, are short to list. The aircraft had no usable height sensor. The detector’s corners were available 59% of the time at the best threshold, in clustered gaps the policy had only just begun training against. The link supported 40 Hz against policies

trained at 60 Hz until the last day. The command path carried, at different times, a $2.4\times$ rate map, an inverted pitch axis on every runner until 21 September, and a yaw sign derived from the wrong message. Three enablement sequences competed in the same week (ACRO with the switch first; ANGLE with arming first; ANGLE in every switch position), each adopted in turn. And the props-off handover check, the one test that proves the abort path works, was deferred on the last day by choice. The team's own final estimate for passing two gates was 30–40%, with the note that nothing in the onboard racing client had ever touched the aircraft.

8. Discussion

Measure the plant before you train against it. The stress study ranked latency and thrust calibration above every other error source before the team reached the hall, and the aircraft then supplied both surprises in exactly that order: a link half as fast as assumed, and a hover throttle that was never measured in free air. Five minutes with a scale and a stopwatch on day one would have been worth more than any of the training runs.

Self-scored accuracy is not accuracy. The auto-labeller's 0.75 px was measured against the model it solved for. Human labels, once available, showed 4.36 px. The same trap appeared in the random train/validation split (0.517 against 0.283) and in the smoke test whose exit code could not fail. In each case the check and the thing checked shared a cause.

A contract that both ends hash is cheap insurance. The observation vector's channel order, sentinels and clips are the one place where a mismatch produces no error, only confident nonsense. The hashed contract, the parity test against the training code and the structural test that shadow mode cannot transmit were the cheapest artefacts in the project and the ones that caught the most.

The failure modes that grounded the fleet were not the ones the simulator studied. Nothing in Figure 8 is a wedged flight controller, a zero stick buffer engaging failsafe, a switch labelled wrong on the transmitter, or an aircraft that arms itself on power-up. Each is documented in the firmware source, each cost an aircraft or a session, and each was preventable by a props-off bench procedure that the team wrote and then, under time pressure, skipped.

Defaults are decisions. The keypoint threshold of 0.5, the reprojection cap of 2 px, the linear stick map, the `np.radians` on raw counts, the mass applied to every link: none was chosen and each was flying. The habit that helped most was reading a value off the artefact rather than the document, and writing down where every number came from with a tag for how much to trust it.

What we would do next time, in order: weigh the aircraft, measure hover throttle and thrust-to-weight from RPM on day one, in the cage; measure the link rate with the stream running and set the simulator's decimation to match before renting a GPU; put a downward range sensor on the Jetson if the rules allow, because every hover design here died for want of height; label a hundred real frames by hand before training anything; and never fly a runner whose handover check has not passed since the last change.

9. Conclusion

Team Electric Fire built a complete vision-only racing system for an aircraft it did not design, in a hall with no tracking, in five days on site after four months of preparation. The simulator, plant model, observation contract, detector and bench measurements are sound and are published. The aircraft

never flew autonomously, and by the end of the qualifier the team had no aircraft left to fly. The gap was not in the learning; it was in the dozen assumptions between the policy’s output and the motors, most of which were measured only after they had cost something. We have written them down so that they need not be measured again.

Acknowledgements

The organizers’ MSP library and camera toolchain on the Jetson made the adapter an afternoon’s work rather than a week’s. Georgia Tech’s AI of Sauron team, working openly on the same airframe, supplied the reference flight-controller configuration and several negative results the team confirmed. The simulator is built on Kousheek Chakraborty’s `isaac_drone_racer`. Most of the on-site code and documents were written with AI coding assistants (Claude, Cursor and Codex), which the commit history records.

References

1. E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature* 620, 982–987, 2023.
2. P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar and D. Scaramuzza, “AlphaPilot: Autonomous drone racing,” *Robotics: Science and Systems*, 2020.
3. S. Li, M. M. O. I. Ozo, C. De Wagter and G. C. H. E. de Croon, “Autonomous drone race: A computationally efficient vision-based navigation and control strategy,” *Robotics and Autonomous Systems* 133, 2020.
4. Y. Song, M. Steinweg, E. Kaufmann and D. Scaramuzza, “Autonomous drone racing with deep reinforcement learning,” *IEEE/RSJ IROS*, 2021.
5. M. Mittal et al., “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE Robotics and Automation Letters* 8(6), 2023 (Isaac Lab).
6. A. Serrano-Muñoz, D. Chrysostomou, S. Bøgh and N. Arana-Arexolaleiba, “skrl: Modular and flexible library for reinforcement learning,” *Journal of Machine Learning Research* 24, 2023.
7. J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
8. J. Eschmann, D. Albani and G. Loianno, “Learning to fly in seconds,” *IEEE Robotics and Automation Letters*, 2024.
9. P. Foehn et al., “Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight,” *Science Robotics* 7(67), 2022.
10. K. Chakraborty, `isaac_drone_racer`, https://github.com/kousheekc/isaac_drone_racer, 2025.
11. G. Jocher, A. Chaurasia and J. Qiu, *Ultralytics YOLOv8*, <https://github.com/ultralytics/ultralytics>, 2023.
12. D. Hafner, J. Pasukonis, J. Ba and T. Lillicrap, “Mastering diverse domains through world models,” arXiv:2301.04104, 2023 (DreamerV3).
13. M. Kelly, C. Sidrane, K. Driggs-Campbell and M. J. Kochenderfer, “HG-Dagger: Interactive imitation learning with human experts,” *IEEE ICRA*, 2019.
14. Betaflight, firmware 4.4.3, <https://github.com/betaflight/betaflight>.

15. Drone Champions League and Anduril, *AI Grand Prix Physical Qualifier Technical Specification*, VADR-TS-005 issue 00.02, 3 September 2026.

Appendix A. Where things are

What	Where
Simulator, plant, observation contract, PPO training, pod scripts	<code>bojro/aigp-sim(contract/, tasks/, dynamics/, utils/, scripts/, docs/)</code>
Auto-labeller, hybrid labeller, dataset tools, training recipes, ONNX deployment, the shipped models	<code>bojro/aigp-perception(aigp_perception/, datasets/, train/, eval/, deploy/, models/)</code>
On-site work: specification and facts, NumPy flight stack, bench tools, measurements, runners, research notes	<code>Code-Red-Cables/AI_GP</code> under <code>pq/</code> (private)
The four eras before the physical qualifier	<code>bojro/AI_GP-archive</code> , sixteen branches (private)
This paper and the script that made its figures	<code>bojro/aigp-sim/paper/</code>

Appendix B. Numbers in one place

Quantity	Value	How it was obtained
Aircraft mass	1745 g	scale, 17 Sep
Camera focal length at 640 px	≈ 425 px (73.7–74.1° HFOV)	three calibrations of the lens
Camera tilt	20° up, locked	set 17 Sep
Telemetry snapshot p95, 40 Hz stream	20.28 ms (budget 25)	300 samples, 21 Sep
Telemetry snapshot p95, 60 Hz stream	30.30 ms (budget 16.7)	same
Gyro scale	16.384 counts per °/s (measured 16.14, 16.25)	integration against attitude
Rate profile	<code>rc_rate 55</code> , super 75, expo 0 $\rightarrow$ 440°/s full stick	<code>MSP_RC_TUNING</code>
Policy rate envelope	3.2 rad/s = 183°/s	action decoder
Linear-map loop gain	0.62 $\times$ at 0.2 rad/s, 2.40 $\times$ at 3.2 rad/s	computed from the profile
Hover stick	0.21–0.29 band in training; 0.233 observed once; 0.30–0.36 on the final airframe	never measured in free air
Detector inference on Orin	27.4–27.6 ms (36 fps), 25 W	<code>bench_torch.py</code>
Keypoint threshold	0.25 (was 0.5 by default): policy ready 59% vs 36%	1345 frames at a gate
Corner dropout model	drop 0.13, sticky 0.81, 4.2% flip rate	same
Labeller accuracy	0.75 px self-scored; 4.36 px median vs humans	340 hand-labelled frames
Fine-tune on hybrid labels	box mAP ₅₀₋₉₅ 0.283 $\rightarrow$ 0.636; pose 0.760 $\rightarrow$ 0.854	contiguous-block split
Split leakage	0.517 random vs 0.283 blocks	same model
Close-gate recall, hand labels	71.9% $\rightarrow$ 100% (32/32)	67 held-out frames

Quantity	Value	How it was obtained
Hover settled, 60 Hz policy	87.0% at 60 Hz, 52.6% at 40 Hz	4096 episodes, 15 s
Hover settled, 40 Hz retrain	85.5% at 40 Hz (p95 0.331 m), 89.0% at 60 Hz	same
Racing at 40 Hz vs 60 Hz	3.09 vs 8.25 gates in 15 s (37% kept)	60 Hz checkpoint
Racing, perfect corners	15.27 gates/episode (race40)	training metric
Racing, corners dropping	10.23 gates/episode (race40drop)	training metric
Pad start to gate 1	0/456 before start fixes; 86.3% after, median 2.15 s	diag_takeoff.py
Full racing run cost	≈48 min, ≈\$0.70 at 4096 envs	RTX 6000 Ada